Estruturas de dados para conjuntos disjuntos

União-e-busca (*union-find*) e a análise do algoritmo de Kruskal

Jair

CMCC – UFABC

Roteiro

1. Onde aparecem os conjuntos disjuntos: o algoritmo de Kruskal
2. O problema e as três operações `cria`, `busca`, `uniao`
3. Representação por **vetor**: simples, mas $\Theta(n)$ por união
4. Representação por **listas ligadas** e a heurística da menor lista
5. Representação por **floresta**: união por tamanho/altura
6. Floresta com **rank** e **compressão de caminhos**
7. Análise: $O((m+n)\lg^* n)$ (e o teorema de Tarjan, $O(m\,\alpha(m,n))$)

Kruskal: onde aparecem os conjuntos disjuntos

Enquanto S evolui de $\emptyset$ até uma árvore geradora, (V, S) é sempre uma **subfloresta geradora** de G. Para $e = \{x, y\} \notin S$:

$$(V, S \cup \{e\}) \text{ é acíclico } \iff x \text{ e } y \text{ estão em componentes conexas distintas de } (V, S).$$

Logo, o teste de aciclicidade do Kruskal equivale a verificar se e não liga vértices do mesmo conjunto da partição de V induzida pelas c.c..

Ao inserir e em S, **unimos** o conjunto de x com o de y.

$\Rightarrow$ precisamos de estruturas que representem e manipulem, dinamicamente, conjuntos disjuntos de modo eficiente: *union-find*.

O problema e as operações

Representar dinamicamente uma família $\mathcal{F} = \{A_1, \ldots, A_k\}$ de subconjuntos disjuntos do universo $\mathcal{U} = \{1, \ldots, n\}$, em que cada conjunto é identificado por um **representante** (elemento do próprio conjunto). Enquanto um conjunto não for modificado por uma união, a resposta sobre o seu representante deve ser sempre a mesma.

`cria`(x) para x que ainda não pertence a nenhum conjunto, acrescenta $\{x\}$ à família $\mathcal{F}$;

`busca`(x) devolve o representante do conjunto ao qual x pertence;

`uniao`(x, y) para x e y em conjuntos distintos A_x e A_y, remove A_x e A_y de $\mathcal{F}$, acrescenta $A_x \cup A_y$ e escolhe um representante para ele.

Como comparar as estruturas

Supomos que as n primeiras operações são `cria`, uma para cada elemento de $\mathcal{U}$, e analisamos o tempo total de uma **sequência de m operações**. Em particular, $m \geq n$.

Kruskal com conjuntos disjuntos

Entrada: grafo conexo $G = (V, E)$ com pesos nas arestas

1: $S \leftarrow \emptyset$
2: $F \leftarrow$ lista das arestas em ordem crescente de peso
3: **para cada** $v \in V$ **faça**
4: `cria`(v)
5: **para cada** $\{u, v\} \in F$ **faça**
6: **se** `busca`$(u) \neq$ `busca`(v) **então**
7: insira $\{u, v\}$ em S
8: `uniao`(u, v)
9: **devolva** (V, S)

Contagem das operações

- $|V|$ operações `cria`;
- no máximo $2|E|$ operações `busca`;
- no máximo $|V| - 1$ operações `uniao`.

Total: menos que $2(|E| + |V|) = O(|E|)$, pois G é conexo e $|V| \leq |E| + 1$.

Consequência

Com $m = O(|E|)$ operações, o custo do Kruskal é

$$O(|E| \log |E|) \text{ (ordenação) } + \text{ custo de } m \text{ operações.}$$

Como $|E| \leq |V|^2$, $O(|E| \log |E|) = O(|E| \log |V|)$. A estrutura escolhida decide se a ordenação domina.

Representação por vetor: a ideia

Vetor C indexado por $\mathcal{U} = \{1, \ldots, n\}$ tal que x, y estão no mesmo conjunto $\iff C[x] = C[y]$. O representante é o **menor elemento** do conjunto: $C[x]$ é o menor elemento do conjunto de x (invariante).

cria(x)

1: $C[x] \leftarrow x$

busca(x)

1: **devolva** $C[x]$

uniao(x, y)

1: $a \leftarrow \texttt{busca}(x)$; $b \leftarrow \texttt{busca}(y)$; $k \leftarrow \min\{a, b\}$
2: **para cada** $i \in \{1, \ldots, n\}$ **faça**
3: **se** $C[i] = a$ **ou** $C[i] = b$ **então**
4: $C[i] \leftarrow k$

O invariante é preservado: k é o menor elemento da união, pois a e b são os menores elementos dos dois conjuntos, e o laço atribui k a $C[i]$ exatamente para os i desses dois conjuntos, sem alterar os demais.
É importante guardar a e b antes do laço: os valores $C[x]$ e $C[y]$ mudam durante ele. `uniao` supõe que todos os elementos já foram criados (o laço lê $C[i]$ para todo i).

Representação por vetor: análise

Custo

`cria` e `busca` custam $O(1)$; `uniao` custa $\Theta(n)$, pois o laço percorre sempre as n posições de C. Como há no máximo $n-1$ uniões (cada uma reduz em 1 o número de conjuntos), m operações custam $O(m + n^2)$.

Corolário (Kruskal com vetor)

O tempo é $O(|E| \log |V| + |V|^2)$ para todo grafo conexo com pesos nas arestas.

Prova.

Ordenar custa $O(|E| \log |E|)$. Há $|V|$ operações `cria` e no máximo $2|E|$ operações `busca`, de custo $O(1)$, e no máximo $|V| - 1$ uniões, de custo $O(|V|)$. O total é $O(|E| \log |E| + |E| + |V|^2)$. Como G é conexo, $|E| \geq |V| - 1$, logo $O(|E| \log |V| + |V|^2)$. □

Gargalo

Em grafos esparsos, $|V|^2$ domina $|E| \log |V|$: precisamos de estruturas com união mais barata.

Listas ligadas: estrutura e operações

Cada conjunto é uma lista ligada cujo primeiro nó é o representante; um vetor $P[1..n]$ guarda, para cada x, um ponteiro para o nó de x (`nil` se x ainda não foi criado). Cada nó tem os atributos `prox` e `rep` (ponteiro para o representante); o nó representante guarda ainda o número de elementos ℓ da lista e um ponteiro `ult` para o último nó.

`cria`(x) cria um nó z com $\texttt{rep}[z] = z$, $\texttt{prox}[z] = \texttt{nil}$, $\ell = 1$, $\texttt{ult} = z$, e faz $P[x] \leftarrow z$;

`busca`(x) devolve $\texttt{rep}[P[x]]$, em tempo $O(1)$;

`uniao`(x, y) sejam L_x e L_y as listas de x e y. Percorre-se L_y atualizando o `rep` de cada nó para o representante de L_x, liga-se $\texttt{ult}[L_x]$ ao primeiro nó de L_y e atualizam-se ℓ e `ult` do representante de L_x, que passa a ser o da união. Custo: $O(1 + \ell[L_y])$.

Uma busca é feita em tempo constante e o custo de uma união é basicamente o custo das atualizações de `rep`. Sem outra precaução, $n - 1$ uniões podem custar $\Theta(n^2)$ (basta sempre atualizar a lista maior): o mesmo que o vetor.

Listas ligadas: a heurística da menor lista

Atualizar sempre a menor lista (‡)

Numa união, os ponteiros `rep` são atualizados na menor das duas listas (em qualquer uma, se têm o mesmo tamanho), que é acrescentada ao final da maior; isto é, a menor lista faz o papel de L_y na descrição de `uniao`.

Intuição: cada vez que `rep`$[x]$ é atualizado, a lista de x pelo menos dobra de tamanho.

Proposição

Supondo (‡), em qualquer momento, se `rep`$[x]$ foi atualizado k vezes, então a lista que contém x tem pelo menos 2^k elementos.

Prova.

Os conjuntos só crescem, logo o tamanho da lista de x não diminui. Indução em k: para $k = 0$, a lista tem pelo menos $1 = 2^0$ elemento. Seja $k \geq 1$ e L a lista de x imediatamente antes da k-ésima atualização; por indução e monotonicidade, $|L| \geq 2^{k-1}$. Como `rep`$[x]$ é atualizado, por (‡) a outra lista L' tem $|L'| \geq |L|$, e a lista resultante tem $|L| + |L'| \geq 2|L| \geq 2^k$ elementos. □

Listas ligadas: resultados

Corolário

Supondo (‡), o representante de um elemento qualquer muda no máximo $\lfloor \log_2 n \rfloor$ vezes.

Prova. Se muda k vezes, a lista de x tem pelo menos 2^k elementos e no máximo n, logo $2^k \leq n$.

Teorema

Supondo (‡) e que as n primeiras operações são `cria`, o tempo para m operações num universo com n elementos é $O(m + n \log n)$.

Prova. `cria` e `busca` custam $O(1)$, e `uniao` custa $O(1)$ mais o número de ponteiros `rep` atualizados. Cada um dos n elementos tem `rep` atualizado no máximo $\lfloor \log_2 n \rfloor$ vezes: o total de atualizações é $O(n \log n)$.

Corolário (Kruskal com listas ligadas e (‡))

Para todo grafo simples conexo $G = (V, E)$ com $|V| \geq 2$, o tempo é $O(|E| \log |V|)$.

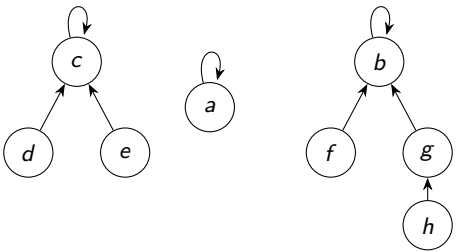
Prova. Ordenar custa $O(|E| \log |E|) = O(|E| \log |V|)$. São $m = O(|E|)$ operações; pelo teorema com $n = |V|$ custam $O(|E| + |V| \log |V|)$, e como $|V| \leq |E| + 1$ e $|E| \geq 1$ o total é $O(|E| \log |V|)$.

Floresta: cada conjunto é uma árvore

Cada elemento é um nó que aponta para o seu pai; a raiz é o representante e o pai da raiz é ela mesma.

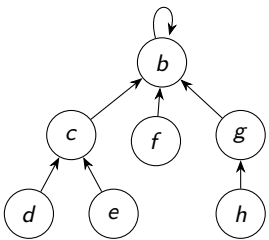
- `cria` cria, em tempo constante, uma árvore com um só nó;
- `busca`(x) percorre o caminho de x até a raiz, em tempo proporcional à profundidade de x, e devolve a raiz;
- a união de duas árvores é feita em tempo constante *a partir das duas raízes*, pendurando uma na outra.

Portanto, `uniao`(x, y) custa duas buscas mais $O(1)$.



Representação com árvores de $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$.

Floresta: o problema e a união por tamanho



Resultado de `uniao`(e, f): a raiz c (tamanho 3) é pendurada na raiz b (tamanho 4).

Essa estrutura pode ser muito ruim: se pendurarmos sempre a raiz da árvore maior numa árvore unitária, formamos uma cadeia com n nós, em que cada busca no nó mais profundo custa $\Theta(n)$.

Floresta: o problema e a união por tamanho

O *tamanho* de uma árvore é o seu número de nós.

A *altura* de uma árvore é a maior distância de um nó até a raiz.

União por tamanho

A raiz da árvore de menor tamanho é pendurada na raiz da árvore de maior tamanho (em caso de empate, em qualquer uma).

Qual a relação entre tamanho e altura após uma sequência de uniões com essa heurística?

Floresta: algoritmos com união por tamanho

Cada nó tem $\mathrm{pai}[x]$ e, nas raízes, $t[x]$ = número de elementos da árvore enraizada em x.

cria(x)

1: $\mathrm{pai}[x] \leftarrow x$
2: $t[x] \leftarrow 1$

busca(x)

1: **enquanto** $x \neq \mathrm{pai}[x]$ **faça**
2: $\quad x \leftarrow \mathrm{pai}[x]$
3: **devolva** x

link(x, y)

1: $t[x] \leftarrow t[x] + t[y]$
2: $\mathrm{pai}[y] \leftarrow x$

`link` pressupõe que x e y são raízes distintas com $t[x] \geq t[y]$ (garantido por `uniao`); `uniao` pressupõe que x e y estão em árvores distintas.

uniao(x, y)

1: $r_x \leftarrow$ `busca`(x); $\quad r_y \leftarrow$ `busca`(y)
2: **se** $t[r_x] \geq t[r_y]$ **então**
3: $\quad$ `link`(r_x, r_y)
4: **senão**
5: $\quad$ `link`(r_y, r_x)

União por tamanho: tamanho × altura

Proposição

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos 2^h nós, onde h é a sua altura.

União por tamanho: tamanho × altura

Proposição

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos 2^h nós, onde h é a sua altura.

Prova.

Indução no número k de uniões já realizadas. Para $k = 0$, as únicas operações até então que alteram a estrutura são `cria`, e toda árvore tem um nó e altura 0, com $1 \geq 2^0$. Seja $k \geq 1$: como as n primeiras operações são `cria`, nada além das uniões altera a estrutura. A k-ésima união liga duas árvores distintas A e B, com tamanhos t_A, t_B e alturas h_A, h_B; por indução $t_A \geq 2^{h_A}$ e $t_B \geq 2^{h_B}$. Sem perda de generalidade $t_A \geq t_B$ e a raiz de B é pendurada na de A. A árvore resultante tem $t' = t_A + t_B$ nós e altura $h' = \max\{h_A, h_B + 1\}$.

- Se $h' = h_A$: $t' \geq t_A \geq 2^{h_A} = 2^{h'}$.
- Se $h' = h_B + 1$: $t' = t_A + t_B \geq 2t_B \geq 2 \cdot 2^{h_B} = 2^{h'}$.

□

União por tamanho: custo

Corolário

Usando união por tamanho, com as n primeiras operações `cria`, o tempo de m operações é $O(m(1 + \log n))$.

Prova.

Pela proposição, uma árvore de altura h tem $2^h \leq n$ nós, logo $h \leq \log_2 n$ e cada `busca` custa $O(1 + \log n)$. Cada `cria` custa $O(1)$ e cada `uniao` custa duas buscas mais $O(1)$, isto é, $O(1 + \log n)$. O total é $O(n + m(1 + \log n)) = O(m(1 + \log n))$, pois $m \geq n$. □

Kruskal com floresta

Com $m = O(|E|)$ e $n = |V|$, o custo das operações é $O(|E| \log |V|)$; somado à ordenação, o tempo total é $O(|E| \log |V|)$.

União por altura e por *rank*

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura.

União por altura e por *rank*

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura.

União por altura

A raiz da árvore mais baixa é pendurada na raiz da árvore mais alta (em caso de empate, em qualquer uma).

Cada raiz r guarda $h[r]$, a altura da árvore enraizada em r, iniciada com 0. `link` pressupõe raízes distintas com $h[x] \geq h[y]$.

cria(x)

1: $\mathrm{pai}[x] \leftarrow x$
2: $h[x] \leftarrow 0$

link(x, y) **(por altura)**

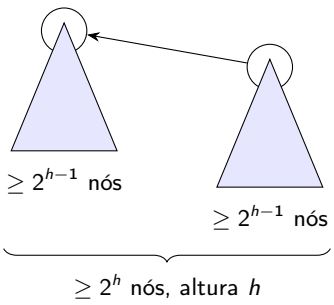
1: **se** $h[x] = h[y]$ **então**
2: $\quad h[x] \leftarrow h[x] + 1$
3: $\mathrm{pai}[y] \leftarrow x$

uniao(x, y)

1: $r_x \leftarrow$ `busca`(x);
2: $r_y \leftarrow$ `busca`(y)
3: **se** $h[r_x] \geq h[r_y]$ **então**
4: $\quad$ `link`(r_x, r_y)
5: **senão**
6: $\quad$ `link`(r_y, r_x)

Floresta: por que a altura fica pequena?

Ideia: a altura só aumenta quando pendurada uma árvore de *mesma altura*. Portanto, uma árvore de altura h foi montada a partir de **duas de altura** $h-1$:



Logo uma árvore de altura h tem $\geq 2^h$ nós; com no máximo n nós, a altura é $\leq \log_2 n$. Cada `busca` custa $O(\log n)$ e

União por altura e por *rank*

Valem os mesmos resultados:

Proposição

Usando união por altura, e supondo que as n primeiras operações são `cria`, em qualquer momento cada árvore tamanho$\geq 2^h$ nós, onde h é a sua altura.

Corolário

Usando união por altura, com as n primeiras operações `cria`, o tempo de m operações é $O(m(1 + \log n))$.

Busca com compressão de caminhos e união por *rank*

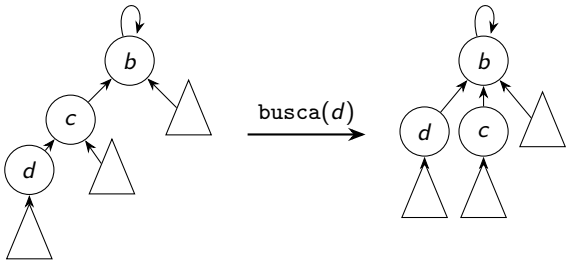
Quando introduzirmos compressão de caminhos, h deixa de ser a altura exata: passa a se chamar **rank**.

Busca com compressão de caminhos e união por *rank*

Quando introduzirmos compressão de caminhos, h deixa de ser a altura exata: passa a se chamar **rank**.

Compressão de caminhos

Quando fazemos `busca`(x), aproveitamos o percurso de x até a raiz e desviamos os ponteiros `pai`$[y]$, de todo y nesse caminho, para a raiz; na união penduramos a árvore de menor *rank* na raiz da de maior *rank*.



Esquema de uma busca com compressão de caminhos: d e c passam a ser filhos da raiz b.

Rank e os algoritmos

A compressão encurta caminhos, e manter a altura exata a cada busca seria caro. Em vez disso guardamos em cada raiz um **rank**:

- começa em 0 e só muda numa união: se as duas raízes têm o mesmo rank, a que recebe sobe 1;
- é uma **cota superior** da altura (igual a ela quando não há compressão);
- um nó de rank r tem pelo menos 2^r descendentes, como no argumento da altura: então $\mathrm{rank} \leq \log_2 n$.

Duas propriedades que usaremos

(a) Todo ancestral próprio de x tem rank **estritamente maior** que o de x.

(b) Depois que deixa de ser raiz, o rank de um nó **nunca mais muda**; o rank do *pai* de x só pode aumentar (ou ficar igual) ao longo do tempo.

Rank e os algoritmos

cria(x)

1: $\text{pai}[x] \leftarrow x$
2: $\text{rank}[x] \leftarrow 0$

busca(x) **com compressão**

1: **se** $x \neq \text{pai}[x]$ **então**
2: $\quad \text{pai}[x] \leftarrow \texttt{busca}(\text{pai}[x])$
3: **devolva** $\text{pai}[x]$

link(x, y)

1: **se** $\text{rank}[x] = \text{rank}[y]$ **então**
2: $\quad \text{rank}[x] \leftarrow \text{rank}[x] + 1$
3: $\text{pai}[y] \leftarrow x$

uniao(x, y)

1: $r_x \leftarrow \texttt{busca}(x)$; $r_y \leftarrow \texttt{busca}(y)$
2: **se** $\text{rank}[r_x] \geq \text{rank}[r_y]$ **então**
3: $\quad \texttt{link}(r_x, r_y)$
4: **senão**
5: $\quad \texttt{link}(r_y, r_x)$

`link` pressupõe que x e y são raízes distintas com $\text{rank}[x] \geq \text{rank}[y]$ (garantido por `uniao`), e `uniao` que x e y estão em árvores distintas.

O resultado de Tarjan e a função α

Teorema (Tarjan, 1975)

Com união por rank e compressão de caminhos, o tempo gasto com m operações num universo de tamanho n, com $m \geq n$ (as n primeiras sendo `cria`), é $O(m\,\alpha(m,n))$.

Não demonstraremos esse teorema (veja CLR); o resultado que mostraremos é um pouco menos preciso.

A função de Ackermann (versão de CLR), para $i, j \geq 1$:

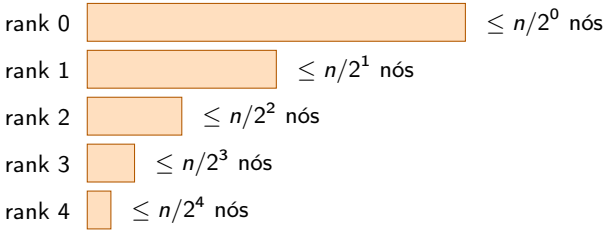
$$A(i,j) = \begin{cases} 2^j & \text{se } i = 1, \\ A(i-1,2) & \text{se } i \geq 2 \text{ e } j = 1, \\ A\big(i-1, A(i,j-1)\big) & \text{se } i \geq 2 \text{ e } j \geq 2, \end{cases} \qquad \alpha(m,n) = \min\big\{ i \geq 1 \colon A(i, \lfloor m/n \rfloor) > \log_2 n \big\}.$$

$A(2,j)$ é uma torre de exponenciais com $j+1$ dois, $A(3,1) = 16$ e $A(4,1) = A(2,16)$ é uma torre de 17 dois, imensamente maior que 10^{80}. Como A é crescente em j, $\alpha(m,n) \leq \alpha(n,n)$, e portanto $\alpha(m,n) \leq 4$ para todo $n < 2^{A(4,1)}$.

n	$\alpha(n,n)$
$1-3$	1
$4-15$	2
$16-65535$	3
$65536 - 2^{A(4,1)}-1$	4

Análise (1/3): poucos nós têm rank alto

Um nó de rank r "traz junto"[1] pelo menos 2^r elementos, e esses elementos **não são reaproveitados** por outro nó do mesmo rank. Logo, há no máximo $n/2^r$ nós de rank r:



É uma **pirâmide**: metade dos nós tem rank baixo, e os ranks altos são raros. Em particular, os nós de rank maior que j são no máximo $n/2^j$.

[1] árvore **no momento em que ele deixa de ser raiz** (depois disso a compressão pode encolher a subárvore, mas esse conjunto fica fixo).

Análise (2/3): agrupar os ranks em faixas

Considere m operações, as n primeiras `cria`, com união por *rank* e compressão. Sejam

$$b_0 = 1 \text{ e } b_g = 2^{b_{g-1}}$$

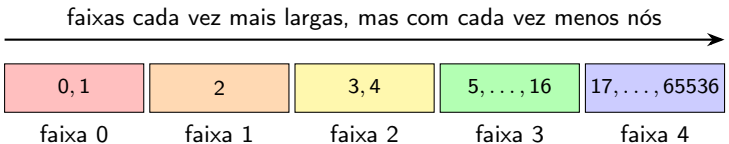
isto é, $1, 2, 4, 16, 65536, \ldots$

Particionamos $\mathcal{U}$ em **grupos** segundo o *rank final*:

- Grupo 0: elementos de rank 0 e 1;
- Grupo 1: elementos de rank em $(1, 2]$;
- Grupo 2: elementos de rank em $(2, 4]$;
- Grupo 3: elementos de rank em $(4, 16]$;
- Grupo 4: elementos de rank em $(16, 65536]$;
- $\vdots$
- Grupo $g \geq 1$: elementos de rank em $(b_{g-1}, b_g] = (K, 2^K]$.

Análise (2/3): agrupar os ranks em faixas

Dividimos os ranks possíveis $(0, \ldots, \log_2 n)$ em **faixas** que ficam cada vez mais largas: a faixa g cobre os ranks $(K, 2^K]$, com K o fim da faixa anterior.



A largura **explode** (de K para 2^K), então bastam $\lg^* n$ faixas para cobrir todos os ranks.

Compromisso: faixa mais larga $\Rightarrow$ um nó dela pode ser contado em mais buscas (o pai dele passa por mais ranks sem sair da faixa), mas a pirâmide garante que há menos nós nela.

Análise (3/3): que nós são contados em cada busca?

O custo de uma busca é o número de nós do caminho percorrido: **contamos $O(1)$ para cada nó**. Separamos os nós do caminho em dois tipos e limitamos quantas vezes cada tipo pode ser contado:

Tipo 1: a raiz, o filho da raiz, ou quem **troca de faixa** para o pai

No caminho o rank só cresce, então o caminho cruza no máximo $\lg^* n$ faixas: $\leq \lg^* n + 1$ **nós por busca**, em $O(m \lg^* n)$ ao todo.

Tipo 2: y e o pai estão na **mesma** faixa

Cada vez que y é contado como Tipo 2, ele passa a apontar para a raiz, que tem rank **maior** que o do pai anterior. Como a faixa tem no máximo 2^K ranks, y é contado no máximo 2^K vezes e depois seu pai sai da faixa. A faixa tem no máximo $n/2^K$ nós:

$$\underbrace{\frac{n}{2^K}}_{\text{nós}} \cdot \underbrace{2^K}_{\text{vezes contado por nó}} = n \qquad \Rightarrow \qquad \text{cada faixa custa } n.$$

Com $\lg^* n$ faixas: $O(n \lg^* n)$.

Resumo: m operações (n primeiras `cria`) e Kruskal

Estrutura	**Tempo de m operações**	**Kruskal**
Vetor	$O(m + n^2)$	$O(\|E\| \log \|E\| + \|V\|^2)$
Listas ligadas + menor lista	$O(m + n \log n)$	$O(\|E\| \log \|V\|)$
Floresta, união por tamanho/altura	$O(m(1 + \log n))$	$O(\|E\| \log \|V\|)$
Floresta, rank + compressão	$O((m + n) \lg^* n)$ (Tarjan: $O(m\,\alpha(m, n))$)	$O(\|E\| \log \|V\|)$

Com listas ou florestas, a ordenação das arestas passa a dominar o custo do Kruskal; a estrutura de dados deixa de ser o gargalo.

Uma justificação mais eleborada do $O(m \log^* n)$

Propriedades do rank

Por construção, o rank de um nó é exatamente a altura da subárvore dele. Em particular, subindo por um caminho, os ranks **crescem estritamente**.

Propgriedade 1

Para todo x: $\mathrm{rank}[x] < \mathrm{rank}(\mathrm{pai}[x])$.

Uma raiz de rank k nasce da fusão de duas árvores cujas raízes têm rank $k-1$.

Propriedade 2

Toda raiz de rank k tem pelo menos 2^k nós em sua árvore.

Propriedade 3

Se há n elementos no total, existem no máximo $n/2^k$ nós de rank k.

Por que valem as propriedades 2 e 3

Propriedade 2 (indução em k).

- Base: rank $0 \Rightarrow$ ao menos $2^0 = 1$ nó (a própria raiz).
- Passo: uma raiz de rank k surge ao unir duas raízes de rank $k-1$, cada uma com $\geq 2^{k-1}$ nós. Logo, $\geq 2 \cdot 2^{k-1} = 2^k$.

Vale também para nós internos. Todo nó interno já foi raiz um dia, e desde então nem o rank nem o conjunto de descendentes dele mudaram. Assim, um nó de rank k tem $\geq 2^k$ descendentes.

Propriedade 3. Dois nós distintos de rank k não têm descendentes em comum: pela Propriedade 1, cada elemento tem no máximo um ancestral de rank k. Os conjuntos de descendentes são disjuntos, cada um com $\geq 2^k$ elementos, então há no máximo $n/2^k$ deles.

Obs.

Precisamos analisar sequências de operações a partir da estrutura vazia e calcular o custo médio (**amortizado**) por operação.

Divisão de trabalho:

- `busca` só mexe no *interior* das árvores;
- `união` só olha para o *nível superior*, formado pelas raízes.

Logo, a compressão não afeta `união` e não altera as raízes.

Ranks são preservados. Quando um nó deixa de ser raiz, *nunca mais* volta a ser, e o rank dele fica fixo para sempre. Os ranks deixam de ser alturas, mas as Propriedades 1–3 continuam valendo.

Intervalos de rank e o $\log^*$

Os ranks dos nós vão de 0 a $\log n$.
Dividimos a parte não nula em intervalos cuidadosamente escolhidos:

$$\{1\},\ \{2\},\ \{3,4\},\ \{5,\ldots,16\},\ \{17,\ldots,2^{16}\},\ \{2^{16}+1,\ldots,2^{65536}\},\ \ldots$$

Cada grupo tem a forma $\{K+1, k+2, \ldots, 2^K\}$, com K potência de 2.

O número de grupos é $\log^* n$: é o número de vezes que é preciso aplicar log a n para chegar a 1 (ou menos).
Na prática só os cinco primeiros intervalos aparecem: os demais exigiriam $n \geq 2^{65536}$.

A "mesada" de cada nó

Estratégia (*contabilidade criativa*): damos a cada nó uma quantia de dinheiro tal que o total distribuído seja $\leq n \log^* n$.

Cada busca custa $O(\log^* n)$ passos, mais uma parcela paga com a mesada dos nós envolvidos (\$1 por unidade de tempo).

Regra da mesada

Um nó recebe sua mesada assim que deixa de ser raiz (quando o rank fica fixo). Se o rank dele está em $\{k+1, \ldots, 2^k\}$, ele recebe 2^k dinheiros.

Total distribuído: $\leq n \log^* n$

Pela Propriedade 3, o número de nós com rank $> k$ é limitado por

$$\frac{n}{2^{k+1}} + \frac{n}{2^{k+2}} + \cdots \leq \frac{n}{2^k}.$$

- Cada um deles recebe 2^k dinheiros, então os nós do intervalo $\{k+1, \ldots, 2^k\}$ recebem no total

$$\leq \frac{n}{2^k} \cdot 2^k = n \text{ dinheiros.}$$

- Há $\log^* n$ intervalos.

$$\boxed{\text{Total distribuído } \leq n \log^* n}$$

Custo de um `find`

O tempo de uma `busca` é o número de ponteiros seguidos. Ao longo da cadeia até a raiz, os ranks crescem, e cada nó x da cadeia é de um de dois tipos:

1. **rank(pai[x]) está em um intervalo mais alto** que rank[x].
   Há no máximo $\log^* n$ nós assim (os ranks crescem estritamente e há só $\log^* n$ intervalos). Custo: $O(\log^* n)$ por `busca`.
2. **rank(pai[x]) está no mesmo intervalo** que rank[x].
   Esses nós pagam \$1 do próprio bolso pelo seu processamento.

A mesada basta para pagar tudo

Precisamos garantir que a mesada de x cobre *todos* os pagamentos dele ao longo da sequência de `finds`.

Observação crucial

Cada vez que x paga \$1, o pai dele muda para um nó de **rank maior** (compressão de caminhos + Propriedade 1).

- Se o rank de x está em $\{k+1, \ldots, 2^k\}$, depois de no máximo 2^k pagamentos o rank do pai cai em um intervalo mais alto.
- A partir daí x passa a ser do tipo 1 e nunca mais paga.
- Mesada recebida: 2^k dinheiros $\Rightarrow$ suficiente.

Conclusão da análise

Juntando as peças, para m operações `find`:

- parcela *por operação*: $O(\log^* n)$ cada, total $O(m \log^* n)$;
- parcela paga pela mesada: no máximo $n \log^* n$ dinheiros, total $O(n \log^* n)$.

$$\boxed{T(m) \;=\; O(m \log^* n) + O(n \log^* n)}$$